

Make `variant_size` SFINAE friendly

Document #: P3664R0
Date: 2025-03-15
Project: Programming Language C++
Audience: LEWG
Reply-to: Zach Laine
<whatwasthataddress@gmail.com>

Contents

- [1 Rationale](#)
- [2 Relationship to `tuple\_size`](#)
- [3 Design](#)
- [4 Wording](#)
- [5 References](#)

§ 1 Rationale

Consider this example using the pattern matching design proposed in [P2688R5](#).

```
void f(const std::any& a) {  
    a match {  
        int: let i => ...  
        double: let d => ...  
    };  
}
```

The protocol for evaluating alternative patterns is to check for completeness of `variant_size<remove_reference_t<decltype((e))>>`, where `e` is the expression being matched.

So for the code above, `variant_size<const std::any>` is instantiated before trying to use the `try_cast` protocol (which applies to `std::any` in particular). In the current design for `variant_size`, `variant_size` tries to unconditionally define a value nested type, whether that would be well-formed or not. Therefore even instantiating `variant_size<std::any>` at all, even without naming its value nested type, generates a hard error.

§ 2 Relationship to `tuple_size`

When `tuple_size` was introduced, it was not SFINAE friendly. [LWG2770] notes that this caused a problem with structured bindings. This is taken directly from the issue:

Consider:

```
#include <utility>

struct X { int a, b; };
const auto [x, y] = X();
```

This is ill-formed: it triggers the instantiation of `std::tuple_size<const X>`, which hits a hard error because it tries to use `tuple_size<X>::value`, which does not exist. The code compiles if `<utility>` (or another header providing `tuple_size`) is not included.

It seems that we either need a different strategy for decomposition declarations, or we need to change the library to make the `tuple_size` partial specializations for cv-qualifiers SFINAE-compatible.

The latter seems like the better option to me, and like a good idea regardless of decomposition declarations.

This is essentially the same problem, just with a different `_size` template.

§ 3 Design

This paper wants to make the analogous tweak to `variant_size` as was used in the accepted resolution of [LWG2770]. Implementation experience with it has shown the same non-SFINAE limitation is causing similar problems in the implementation.

§ 4 Wording

In 22.6.4 [variant.helper]:

- 2 Let `VS` denote `variant_size<T>` of the cv-unqualified type `T`. ~~Then~~If the expression `VS::value` is well-formed when treated as an unevaluated operand, then each specialization of the template meets the *Cpp17UnaryTypeTrait* requirements ([meta.rqmts]) with a base characteristic of `integral_constant<size_t, VS::value>`. Otherwise, it has no member value.

Access checking is performed as if in a context unrelated to `VS` and `T`. Only the validity of the immediate context of the expression is considered.

[*Note:* The compilation of the expression can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the “immediate context” and can result in the program being ill-formed. — *end note*]

In 17.3.2 [version.syn]:

```
#define __cpp_lib_variant 202306202306?????L // also in <variant>
```

§ 5 References

[LWG2770] Richard Smith. `tuple_size<const T>` specialization is not SFINAE compatible and breaks decomposition declarations.

<https://wg21.link/lwg2770>